

Steel Defect Detection: Comprehensive Architecture & Implementation Report

1. Executive Summary

This project was developed for the Severstal Steel Defect Detection initiative. The goal is to automatically locate and segment surface defects on sheet steel using advanced computer vision techniques. Originally consisting of scattered Jupyter notebooks and ad-hoc scripts, the codebase has been completely refactored into a modular, highly scalable, and production-ready deep learning pipeline. The final system leverages a robust 3-model ensemble, a highly optimized REST API, a modern 'Glassmorphism' web interface, and zero-cost cloud deployment workflows.

2. Machine Learning Methodology

To achieve maximum accuracy and generalization, the system utilizes a heterogeneous ensemble of three state-of-the-art segmentation architectures. Each model is designed to capture different levels of spatial features and contextual information:

- **U-Net with ResNet-34:** Excels at capturing fine-grained spatial details through its symmetric encoder-decoder structure and skip connections.
- **FPN (Feature Pyramid Network) with SE-ResNeXt50:** Highly effective at detecting defects at multiple scales by merging high-level semantic features with low-level spatial features.
- **DeepLabV3+ with EfficientNet-B3:** Utilizes Atrous Spatial Pyramid Pooling (ASPP) to capture multi-scale context, making it particularly strong at segmenting large, amorphous defect patches.

Post-Processing & Noise Reduction:

Raw model probabilities are averaged at the sigmoid level. To eliminate false positives and isolated noise pixels, a deterministic Connected Component Analysis (CCA) is applied. Any predicted defect component with an area smaller than a class-specific pixel threshold (e.g., 300 to 2000 pixels) is automatically discarded. This significantly improves precision.

3. Codebase Architecture & File Responsibilities

The repository follows best practices for production AI systems, strictly separating the machine learning logic, the backend API, and the frontend user interface.

A. Core Machine Learning Module (src/)

- **src/models.py:** The model factory. Responsible for dynamically instantiating the ensemble architectures using the `segmentation\_models\_pytorch` framework.

- **src/dataset.py:** Contains the PyTorch `Dataset` classes. It manages image loading, applies heavy data augmentations via `albumentations`, and parses Kaggle's Run-Length Encoded (RLE) strings into strict binary tensors.
- **src/config.py:** Centralizes all project hyperparameters (batch sizes, learning rates, epochs, image dimensions) using Python `dataclasses`.
- **src/utlis.py:** Houses mathematical utilities, specifically the implementation of the Dice Coefficient metric used to evaluate segmentation overlap.

B. Execution Scripts

- **train.py:** The primary training loop. Built for speed and efficiency, it implements Automatic Mixed Precision (AMP) via `torch.cuda.amp` to reduce VRAM usage and accelerate training times.
- **predict.py:** A standalone offline inference engine. Designed to process batches of local images through the ensemble and output overlay visualizations.

C. Production Application

- **api/app.py:** A lightning-fast REST API built on FastAPI. It loads the multi-gigabyte ensemble into memory asynchronously, exposes a `/api/predict` endpoint for binary image uploads, and serves the static frontend.
- **static/ (HTML/CSS/JS):** The user-facing frontend. Features a modern, 'Glassmorphism' dark-mode aesthetic. It allows users to drag-and-drop steel images and renders the API's segmentation masks directly onto an HTML5 canvas in real-time.

4. Cloud Deployment & Automation

Because the ensemble weights exceed standard Git repository limits (approx. 760 MB), a distributed deployment strategy was implemented using GitHub and the Hugging Face Hub.

- **deploy.py & scratch_hf.py:** Custom automation scripts that authenticate via API tokens to programmatically create GitHub repositories, push code, and upload the heavy `.pth` weights directly to the Hugging Face Hub.
- **app.py (Root):** A native Gradio application designed specifically to run on Hugging Face Spaces free-tier hardware.
- **space_static/index.html:** A highly optimized, responsive static landing page hosted on Hugging Face. It serves as the public face of the project, explaining the architecture and directing users to the live demo.
- **notebooks/02_live_demo_colab.ipynb:** An interactive Google Colab notebook. It dynamically downloads the weights from Hugging Face, starts the FastAPI server, and exposes it to the public internet using Cloudflare Tunnels (`cloudflared`). This provides a 100% free, highly available public demonstration link.

- **generate_colab_nb.py:** A CI/CD-style script that programmatically generates the Colab notebook to ensure the demo always stays perfectly synchronized with the repository code.
- **Dockerfile & docker-compose.yml:** Standardized container definitions. Allows any developer to spin up the entire API, frontend, and environment dependencies locally with a single command.

Conclusion

This project represents a complete, end-to-end machine learning lifecycle. It transitions from raw data exploration and model training to robust API development, premium UI/UX design, and zero-cost distributed cloud deployment. The resulting system is fully prepared for real-world industrial application and peer review.